Introduction to Computer Architecture and Systems

Chapter 5+
From Machine Code to Assembly

Fengkai Wang(Iori Oikawa), Fall 2025 @ Drew University

Previously of CSCI 260…

- Revisit: Counting Character Occurrences, from Flowchart to Machine Code
- Motivation to improve machine code to assembly
- Essential Improvements / New Concepts
  - New Format: Label - Opcode - Operand - Comment
  - Labels
  - Comments
  - Pseudo-Ops

Cover-ups: Bit Conversion

How to make machine code better for programmers?

Example: Multiply a number by 6

```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

New Format

From Opcode + Operand to…

```
Label    Opcode    Operands    ; Comment
```

- Label: Improved Line Indicator
- Opcode: Mnemonics in Words
- Operand: As-Is from Machine Code
- Comment: Information for Human Reference

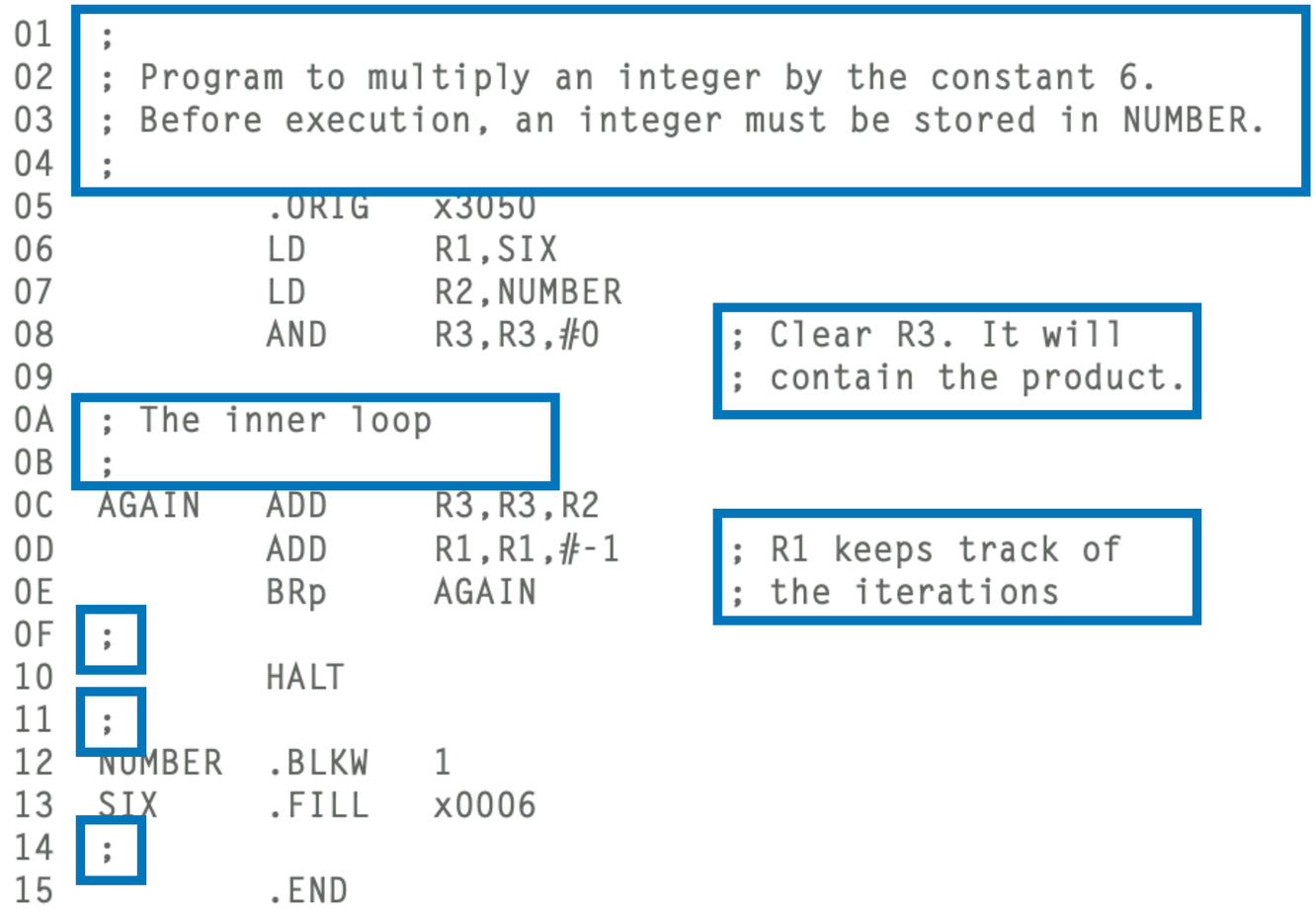
Add-ons: Labels

Instead of counting number of lines, we can use words now

```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Add-ons: Comments

Information starting with ‘;’



```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Add-ons: Pseudo-Ops

All words begin with '.'

```
01  ;
02  ; Program to multiply an integer by the constant 6.
03  ; Before execution, an integer must be stored in NUMBER.
04  ;
05          .ORIG   x3050
06          LD      R1,SIX
07          LD      R2,NUMBER
08          AND     R3,R3,#0        ; Clear R3. It will
09                                  ; contain the product.
0A  ; The inner loop
0B  ;
0C  AGAIN   ADD     R3,R3,R2
0D          ADD     R1,R1,#-1       ; R1 keeps track of
0E          BRp     AGAIN           ; the iterations
0F  ;
10          HALT
11  ;
12  NUMBER  .BLKW   1
13  SIX     .FILL   x0006
14  ;
15          .END
```

Pseudo-Ops

- .ORIG = Set the Program Counter's Initial(ORIGINAL) Position

```
.ORIG   x3050
```

- .BLKW = set aside the next location in the program and initialize it with the value of the operand. The value can be either a number or a label.

```
.BLKW   1
```

- .FILL = set aside some number of sequential memory locations (i.e., a BLocK of Words) in the program. The actual number is the operand of the .BLKW pseudo-op.

```
.FILL   x0006
```

- .END = tells the assembler it has reached the end of the program and need not even look at anything after it. That is, any characters that come after .END will not be processed by the assembler. But .END itself does NOT stop the program.

Pseudo-Ops

- .STRINGZ tells the assembler to initialize a sequence of n + 1 memory locations. The argument is a sequence of n characters inside **double quotation marks**. The first n words of memory are initialized with the zero-extended ASCII codes of the corresponding characters in the string. The final word of memory is initialized to 0. The last word, containing x0000, provides a convenient **sentinel** for processing the string of ASCII codes.

```
        .ORIG     x3010
HELLO   .STRINGZ  "Hello, World!"
```



```
x3010: x0048
x3011: x0065
x3012: x006C
x3013: x006C
x3014: x006F
x3015: x002C
x3016: x0020
x3017: x0057
x3018: x006F
x3019: x0072
x301A: x006C
x301B: x0064
x301C: x0021
x301D: x0000
```

Discussion

How many times will the instruction at the memory address labeled ‘LOOP’ execute?

```
            .ORIG     x3005
             LEA      R2, DATA
             LDR      R4, R2, #0
LOOP         ADD      R4, R4, #-3
             BRzp     LOOP
             TRAP     x25
DATA        .FILL      x000B
            .END
```

Discussion

How many times will the instruction at the memory address labeled ‘LOOP’ execute?

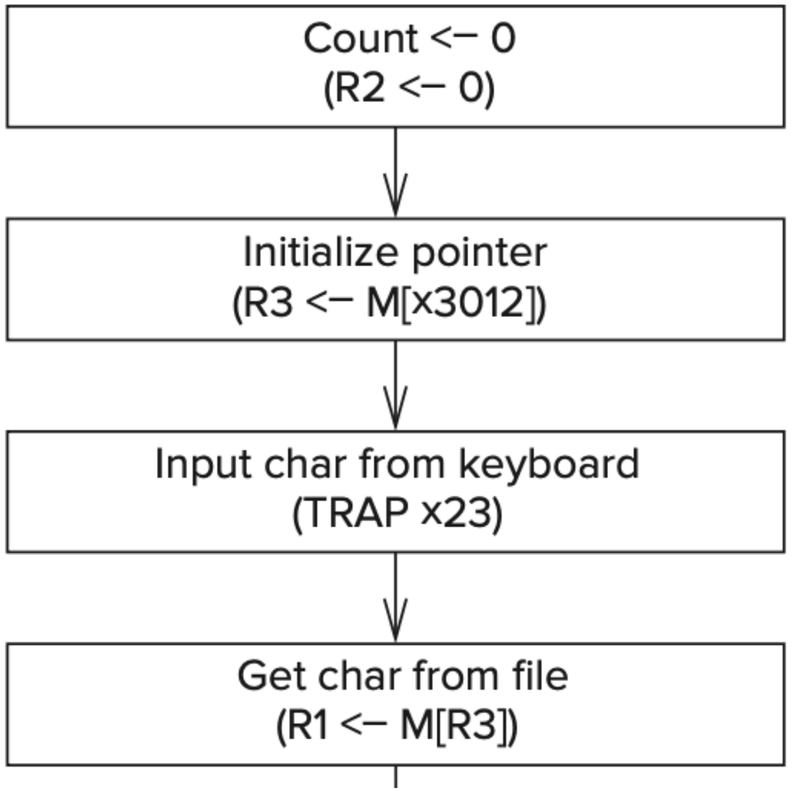
```
        .ORIG    x3005
         LEA     R2, DATA
         LDR     R4, R2, #0
LOOP     ADD     R4, R4, #-3
         BRzp    LOOP
         TRAP    x25
DATA     .FILL    x000B
        .END
```

- R4 starts at 11 (x000B).
- Each pass at LOOP does R4 ← R4 – 3…
- …and then branches back while R4 ≥ 0 (BRzp).

Machine Code to Assembly

Part 1: Preparing

Counter, Pointer, and the Character Looking For

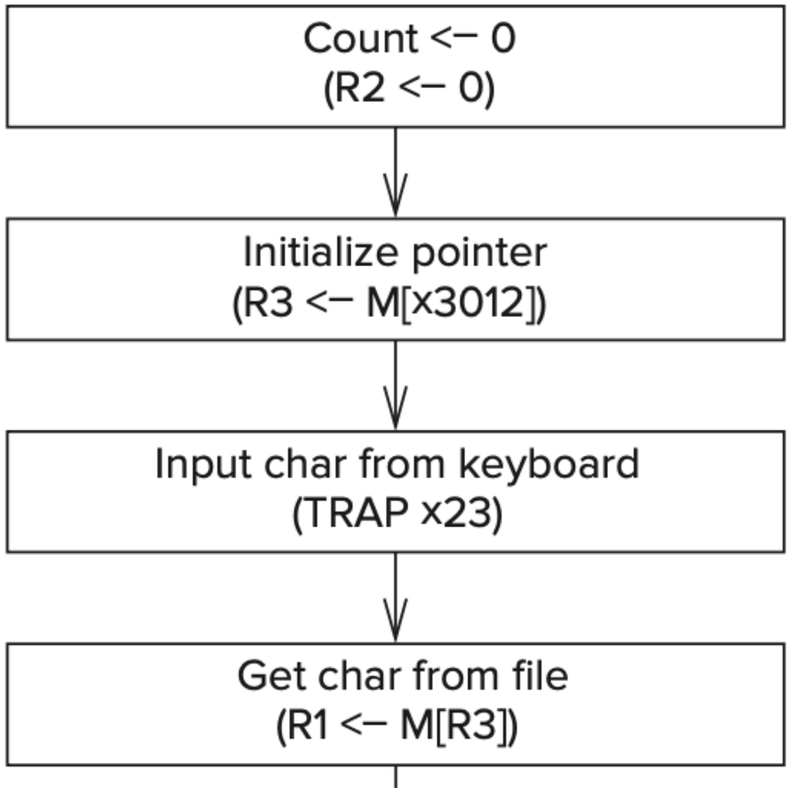


- R2 <- 0
- R3 <- M[3012]
- TRAP x23
- R1 <- M[R3]

- Counter: R2 = 0
- R3 set to begin
- Get input
- Read via Pointer

Part 1: Preparing

Counter, Pointer, and the Character Looking For

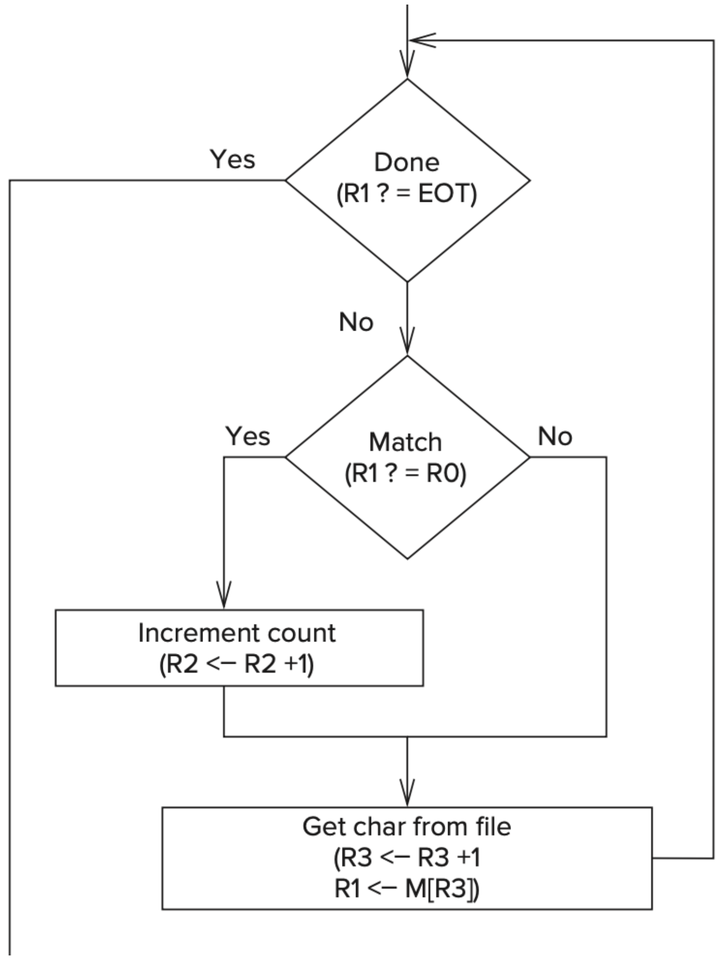


- R2 <- 0
- R3 <- M[3012]
- TRAP x23
- R1 <- M[R3]

- AND R2 R2 0
- LD R3 16
- TRAP x23
- LDR R1 R3 0

Part 2: Process inputs; counting matches

Outside Loop: If Done; Inside Loop: If got match



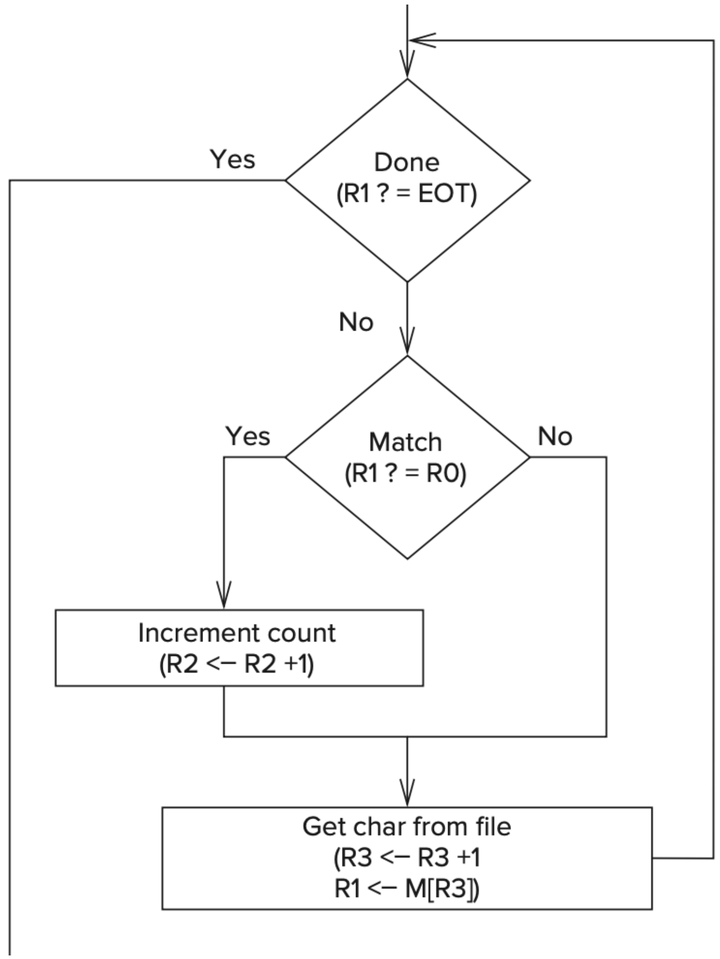
- R4 <- R1 - 4 (EOT)
- BRz x300E
- R1 <- NOT R1
- R1 <- R1 + 1
- R1 <- R1 + R0
- BRnp x300B

- Check EOT
- If so, Part 3
- R1 on negative…

Check if R1 = R0 (Match)
If not, move R3 on

Part 2: Process inputs; counting matches

Outside Loop: If Done; Inside Loop: If got match

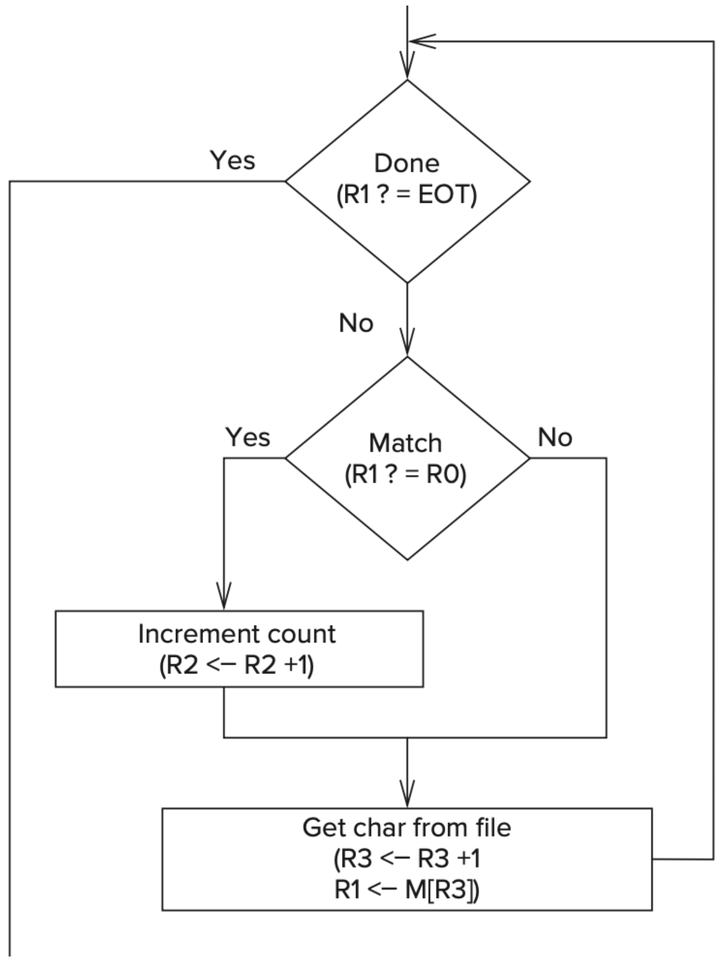


- R4 <- R1 - 4 (EOT)
- BRz x300E
- R1 <- NOT R1
- R1 <- R1 + 1
- R1 <- R1 + R0
- BRnp x300B

- ADD R4 R1 -4
- BRz 8
- NOT R1 R1 11111
- ADD R1 R1 1
- ADD R1 R1 R0
- BRnp 1

Part 2: Process inputs; counting matches

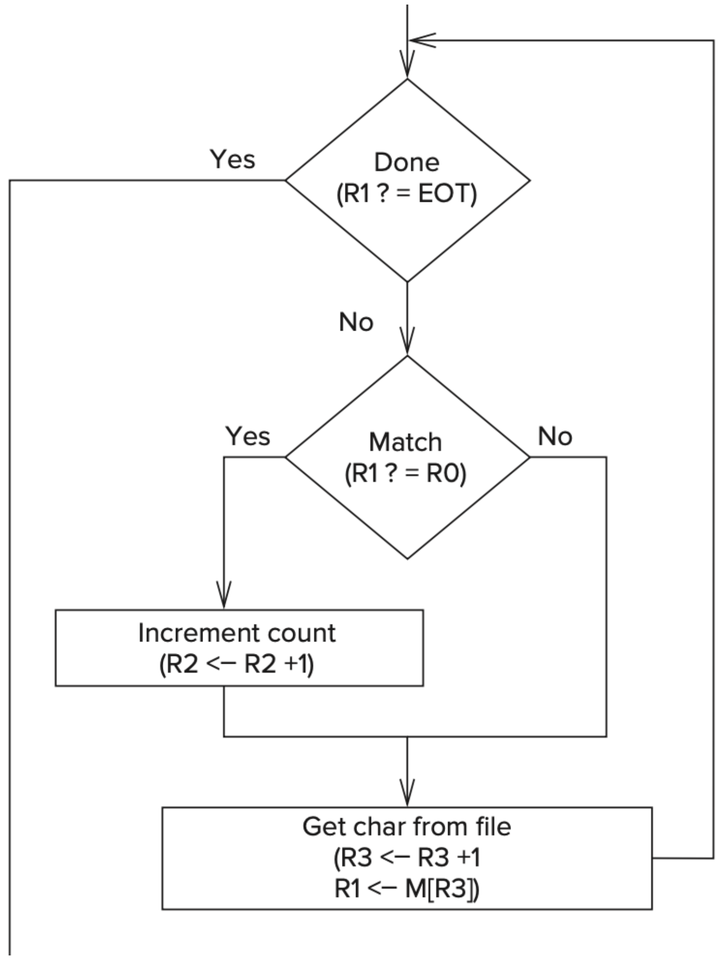
Outside Loop: If Done; Inside Loop: If got match



- R2 <- R2 + 1 If match, add R2…
- R3 <- R3 + 1 And move R3.
- R1 <- M[R3] Load next character
- BRnzp M[x3004] Send back to outer loop

Part 2: Process inputs; counting matches

Outside Loop: If Done; Inside Loop: If got match

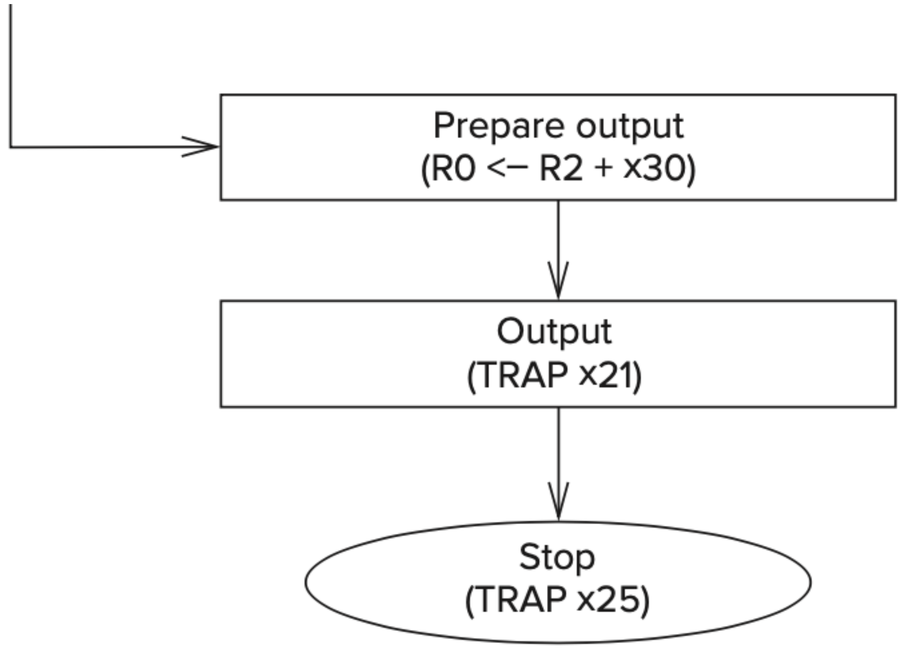


- R2 <- R2 + 1 — ADD R2 R2 1
- R3 <- R3 + 1 — ADD R3 R3 1
- R1 <- M[R3] — LD R1 R3
- BRnzp M[x3004] — BRnzp -10

Counting Occurrence of a Character

Part 3: Prepare Outputs, Finishing Up

If we reached EOT…

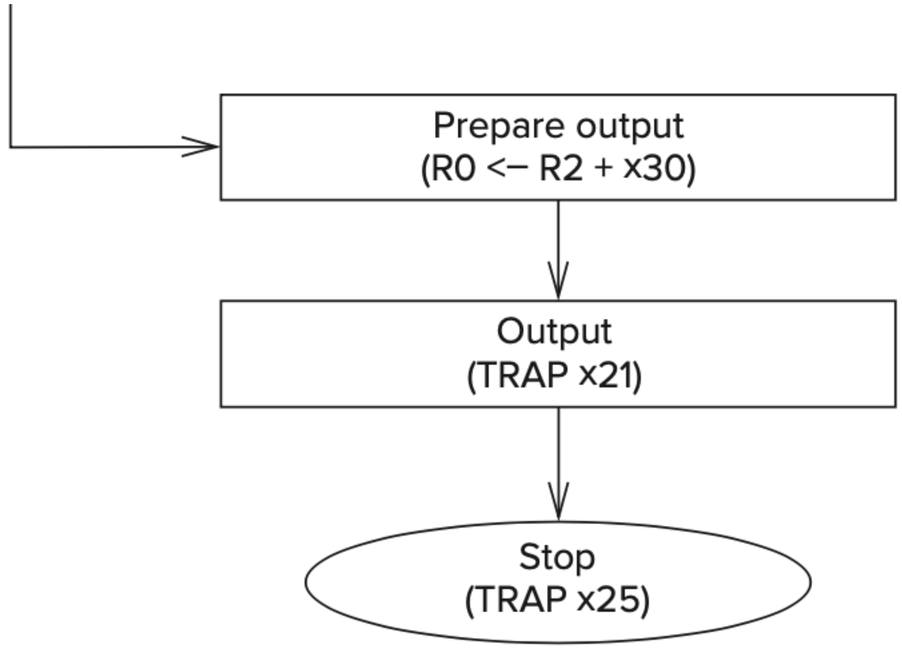


- R0 <- M[x3013] ASCII ‘0’ position
- R0 <- R0 + R2 Counter in ASCII
- TRAP x21 TRAP x21
- TRAP x25 TRAP x25

Counting Occurrence of a Character

Part 3: Prepare Outputs, Finishing Up

If we reached EOT…



- R0 <- M[x3013] — LD R0 4
- R0 <- R0 + R2 — ADD R0 R2
- TRAP x21 — TRAP x21
- TRAP x25 — TRAP x25

Complete Program

	Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
	x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	`R2 <- 0`
	x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0	`R3 <- M[x3012]`
	x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1	`TRAP x23`
	x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	`R1 <- M[R3]`
ASCII x4 ('EOT')	x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	1	0	0	`R4 <- R1-4`
	x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	`BRz x300E`
	x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	`R1 <- NOT R1`
	x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	`R1 <- R1 + 1`
	x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	`R1 <- R1 + R0`
	x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	`BRnp x300B`
	x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	`R2 <- R2 + 1`
	x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	`R3 <- R3 + 1`
	x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	`R1 <- M[R3]`
	x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	`BRnzp x3004`
	x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	`R0 <- M[x3013]`
	x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	`R0 <- R0 + R2`
	x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	`TRAP x21`
	x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	`TRAP x25`
	x3012	Starting address of file																
ASCII x30 ('0')	x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	`ASCII TEMPLATE`

Moving to Assembly

From Machine code

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 <- 0
x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0	R3 <- M[x3012]
x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1	TRAP x23
x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 <- M[R3]
x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	1	0	0	R4 <- R1-4
x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	BRz x300E
x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	R1 <- NOT R1
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 <- R1 + 1
x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	R1 <- R1 + R
x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	BRnp x300B
x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	R2 <- R2 + 1
x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	R3 <- R3 + 1
x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 <- M[R3]
x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	BRnzp x3004
x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	R0 <- M[x3013]
x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	R0 <- R0 + R2
x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	TRAP x21
x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	TRAP x25
x3012	Starting address of file																
x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	ASCII TEMPLATE

```
01   ;
02   ; Program to count occurrences of a character in a file.
03   ; Character to be input from the keyboard.
04   ; Result to be displayed on the monitor.
05   ; Program works only if no more than 9 occurrences are found.
06   ;
07   ;
08   ; Initialization
09   ;
0A            .ORIG   x3000
0B            AND     R2,R2,#0        ; R2 is counter, initialize to 0
0C            LD      R3,PTR          ; R3 is pointer to characters
0D            TRAP    x23             ; R0 gets character input
0E            LDR     R1,R3,#0        ; R1 gets the next character
0F   ;
10   ; Test character for end of file
11   ;
12   ;
13   TEST     ADD     R4,R1,#-4       ; Test for EOT
14            BRz     OUTPUT          ; If done, prepare the output
15   ;
16   ; Test character for match.  If a match, increment count.
17   ;
18            NOT     R1,R1
19            ADD     R1,R1,#1        ; R1 <-- -R1
1A            ADD     R1,R1,R0        ; R1 <-- R0-R1. If R1=0, a match!
1B            BRnp    GETCHAR         ; If no match, do not increment
1C            ADD     R2,R2,#1
1D   ;
1E   ; Get next character from the file
1F   ;
20   GETCHAR ADD     R3,R3,#1        ; Increment the pointer
21            LDR     R1,R3,#0        ; R1 gets the next character to test
22            BRnzp   TEST
23   ;
24   ; Output the count.
25   ;
26   OUTPUT   LD      R0,ASCII        ; Load the ASCII template
27            ADD     R0,R0,R2        ; Convert binary to ASCII
28            TRAP    x21             ; ASCII code in R0 is displayed
29            TRAP    x25             ; Halt machine
2A   ;
2B   ; Storage for pointer and ASCII template
2C   ;
2D   ASCII    .FILL   x0030
2E   PTR      .FILL   x4000
2F            .END
```

Activity Take Home:
More assembly programming